

OOPS (defⁿ & examples are very important) ①

Why OOP?

- Scattered Data
- Bundle Data as a unit (Class)
- Helps model software ^{using} as objects
- Maintainability & Data protection & Code behaviour.
- Reusability ★

4 pillars

- Encapsulation
- Abstraction
- Inheritance
- Polymorphism

Class → structure & entity's behaviour (Blueprint ★)

→ makes code clean (saves creating variables again & again)

OR
group of objects → data & functions

Object → real world entity
→ Class Instance

→ Has own non-static data member

→ Actual memory allocated when object created

★ In STL library implementation, OOPS is used BTS.

★ Object is not properties / functions associated not !!
(or) methods (or) member functions

Class Name {

};

int main ()

{
 Name n1 → object of class
 ✓

class
{
 ✓
}

n1. --- } ⇒ accessing n1 object's property

Access Modifiers

- { private } → class or access restricted !!! (default)
- { public } → everywhere accessible
- { protected } → class & derived class only accessible

☆ ^{give} to access of private properties ^(to int main) use getter setter functions.

Encapsulation (capsule) → Data Hiding (using private/protected)

↳ wrapping up of data & member functions in a class (single unit).

Constructor → always public
 → same name of class
 → no return type

→ special method (auto-invoked)
 → on object-creation (each)
 → used to initialize default values.

Name C)
 {
 }
 } → class ka constructor
 A constructor
 avasth me !!

Types of constructor

- ↳ Parametrized (passing parameters in constructor itself)
- ↳ copy constructor
- ↳ Non parametrized

Constructor overloading

⇒ same name of constructors avasth
 (can happen with different parameters)

this → a pointer (pointing to current object of property)

constructor or parameter

⇒ this → name = name

this → dept = dept

→ object of property

* $\rightarrow$ dereferencing pointer

int x = 10
 Stores 10

int * ptr = &x
 Stores 10 at address

* ptr $\rightarrow$ accessing x itself

* this $\rightarrow$ object access

* this . property $\equiv$ (this $\rightarrow$ property)
 object of property

Revisé pointers

(copy constructors) $\rightarrow$ (default)

$\rightarrow$ copies properties of one object into another

Name n₁ (---, ---, ---)

Name n₂ (n₁); $\rightarrow$ creates a copy of n₁ and puts to n₂.

$\rightarrow$ By default (the way it works!!)

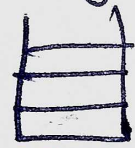
```
Name (Name &x) {
    cout << "I am custom copy constructed";
    this  $\rightarrow$  name = &x . name;
}
```

Shallow copy $\Rightarrow$ one object की other में direct copy करना runtime allocation ⑤

$\hookrightarrow$ creates problem when class के अंदर dynamic memory allocation है !! $\rightarrow$ done by using new keyword

$\hookrightarrow$ Heap के अंदर कुछ हिस्सा है!

Generally normal memory allocation happens in a stack memory.



compile time allocation / statically allocated

why?

जो pointers change की जाती हैं, वे भी change हो जायेंगे ($\because$ अगर -2 से values copy करता है !!)

$\hookrightarrow$ object₂ is pointing to some address

अब object₁ point कर रहा है, hence this problem occurs !!

(constructor)

Deep copy not by default (dynamic allocated memory की भी copy बनती है !!)

$\hookrightarrow$ Heap वाली memory (address) की भी copy बनती है !!

(code)
 $\hookrightarrow$ Has different from shallow copy $\hookrightarrow$ (RAM की copy बनाती है)

★ (See the codes of Deep copy & Shallow copy.) (6)

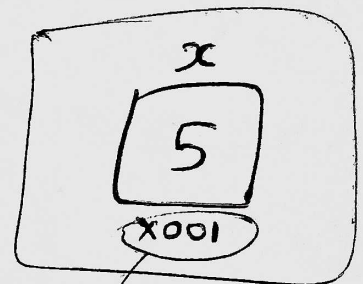
Destructor → Deallocating memory given to object
↳ only deallocates statically allocated memory, not RAM.

new int x = 5

int *ptr = x

delete ptr

↳ deletes memory
of x from heap



say
pointer ptr stores
this address

~Name()

{

cout << "This is a default destructor"

}

added for

deleting DMA that memory

delete var name to delete
↳ auto called as object goes out of scope

(DMA that)

★ Destructor helps save memory leaks (by deleting it)

Inheritance → (code reusability)

↳ property & member function of base class passed to derived class.

Class A {

};

Class B : public A { properties
_____ of class A
_____ publicly!!!

};

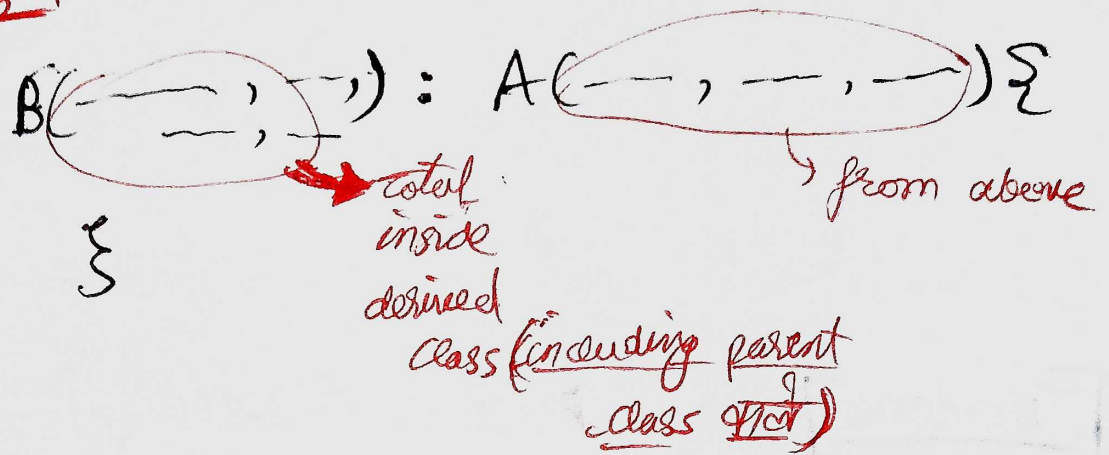
★ when making class B object, first A (Base class) constructed call first, then B!!

↳ if constructor is default

★ when destructor is called and memory is deallocated, then child class deallocate then parent class!!

→ if constructor
is parametrized

②



Mode of Inheritance private
public
protected

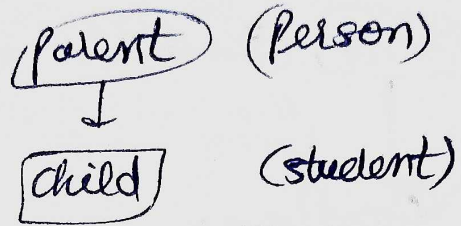
<u>Parent Class</u>	<u>Derived Class</u>		
	Private Mode	Protected M.	Public M.
Private	Not Inherited	N.I.	N.I.
Protected	Private	Protected	Protected
Public	Private	Protected	Public

★ Private property or private members are never inherited !!

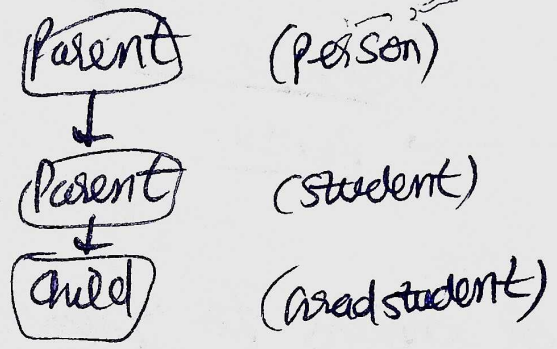
★ If we need to pass property/method just for inheritance, we use protected !!

Types of Inheritance

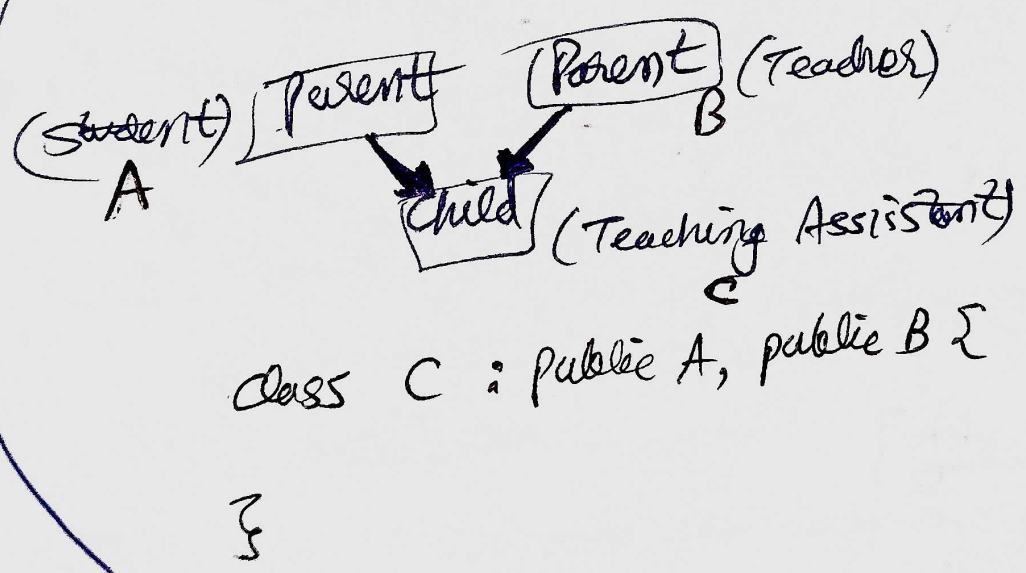
↳ single level



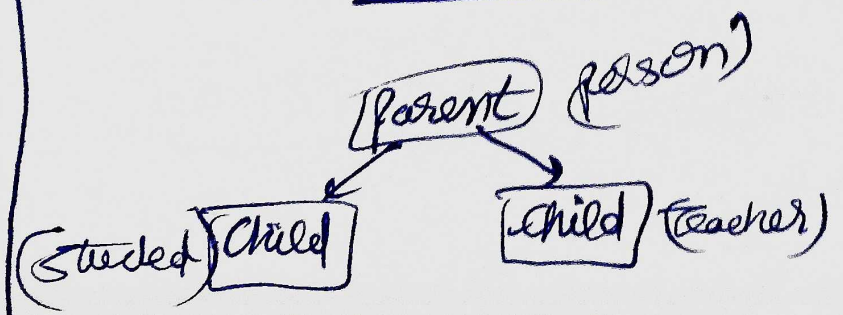
↳ Multi level



↳ Multiple Inheritance

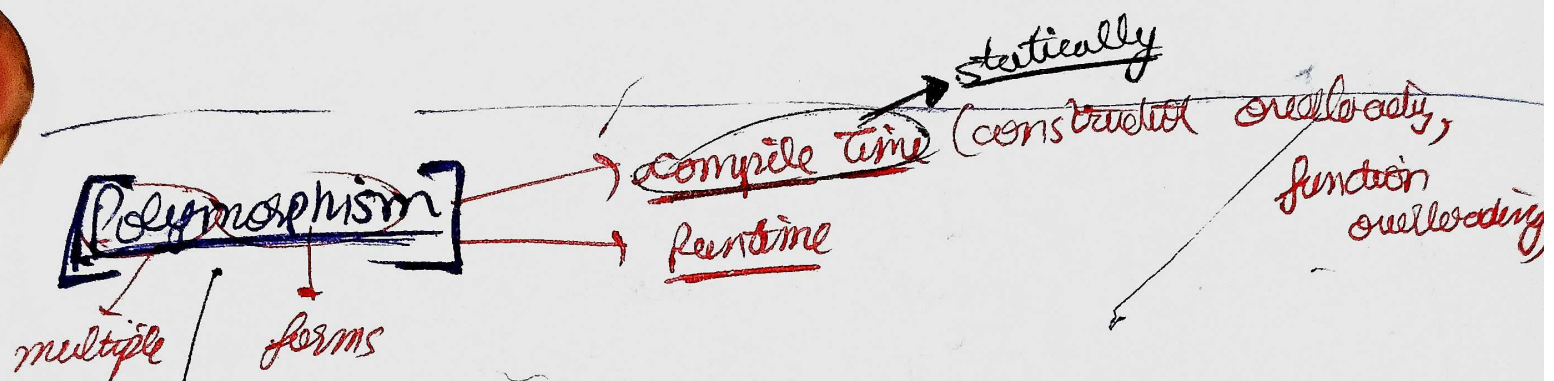
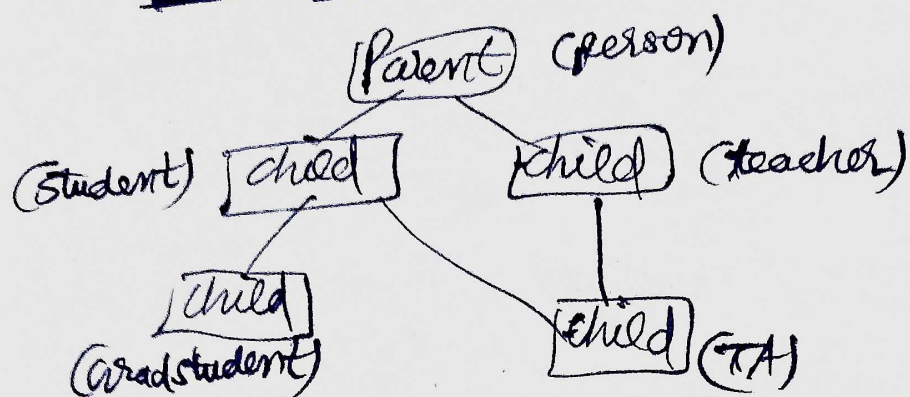


↳ Hierarchical Inheritance



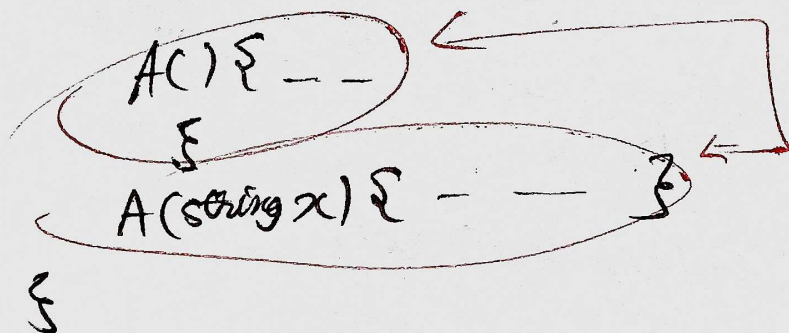
Hybrid Inheritance

(10)



→ ability of objects to take different forms or behave in different ways depending on context.

class A {



constructor overloading
is an example of
polymorphism

Operator overloading

string b = "abc"

string a = b

→ This sym is overloaded, b int copy is created in a!!

Runtime Polymorphism

Dynamic

→ ex: Function overriding → parent, child have same name function, but different use case.

[It is decided on runtime and which class object is made??]

★ If object of child class created → child class function called.

★ ————— parent ————— → parent —————.

★ parent class function is said to be overridden.

→ ex: Virtual Functions → Dynamic in nature
→ defined only in Base/parent class
→ called during runtime

→ A member function that is expected to be redefined or overridden in derived class.

↳ works like overriding

12



(OR) → used to define an interface of derived classes (can't be instantiated)

★ A class with pure virtual function is by default an abstract class, ∴ ~~अ~~ modify न किया जा सकता !!!

Static keyword

- S. variable
- S. object

For lifetime of program
its created and initialized
only once.

★ static variables are shared by all objects of the class.

(13)

☆☆ If there is a static object of some class,
it will run till lifetime of the
program. (or, call to destructor is made
after everything is executed).
